

INSTITUTO FEDERAL DO CEARÁ
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

BRUNO ALENCAR ALVES

Funções Avançado



IGUATU-CE
2025

1. Introdução

I. Definição

Definição: uma função é um bloco de código que executa uma tarefa específica quando chamado. Ela é projetada para receber entradas, processá-la conforme as instruções definidas dentro da função e, opcionalmente, retornar um resultado.

Componentes de uma função:

1. Identificador.
2. Chamada.
3. Parâmetros.
4. Corpo.
5. Valor de retorno.

Sintaxe de declaração:

```
1. function identificador(parâmetro1, parâmetro2, ...){  
2.     bloco de código;  
3.     return valor de retorno;  
4. }  
5.  
6. identificador(arg1, arg2, ...);
```

function hoisting: em Javascript hoisting é um comportamento padrão onde funções declaradas com “function” são elevadas para o topo do código durante a fase de compilação.

Objeto de primeira classe: em JavaScript, função são consideradas objetos de primeira classe, ou seja, podem ser atribuídas a variáveis, serem passadas como argumento para outras funções, servirem de retorno de outras funções e serem armazenadas em estruturas de dados como arrays.

2. Parâmetros de funções

I. Definição

Definição: parâmetros são variáveis declaradas entre parênteses na definição da função. Eles recebem valores chamados de argumentos quando a função é chamada.

Sintaxe:

```
1. function exemplo (parâmetro1, parâmetro2, parâmetron,...){  
2.     //bloco de código;  
3. }  
4.  
5. exemplo(valor1,valor2,valor3,..);
```

Observações

Obs1: Funções declaradas com function podem receber um número maior de argumentos do que o número de parâmetros declarados na função.

Obs2: Funções declaradas com function quando for passar um número menor de argumentos do que o número de parâmetros definido na função, os valores dos parâmetros serão inicializados com undefined.

Obs3: Ao trabalhar com funções existe uma variável chamadas arguments que recebe todos os argumentos passado para a função.

II. Valor padrão para parâmetros.

Conceito: caso exista a possibilidade de não ser passado nem um argumento para algum parâmetro da função é possível definir um valor padrão para que o mesmo não seja inicializado como undefined.

Sintaxe:

```
1. function exemplo (parâmetro1 = valor1, parâmetro2 = valor2){  
2.   //bloco de código;  
3. }  
4.  
5. exemplo();
```

III. Acessando valores padrões.

Conceito: caso queira acessar valores padrões de uma função definida com mais de um parâmetro, é preciso passar como argumento undefined.

Sintaxe:

```
1. function exemplo (parâmetro1 = valor1, parâmetro2 = valor2, parâmetro3 = valor3){  
2.   //bloco de código;  
3. }  
4.  
5. exemplo(undefined,undefined,100);
```

IV. Atribuição via destruturação com funções para arrays.

Conceito: é possível pegar os valores de um array e atribuí-los aos parâmetros individuais de uma função, basta que eu pegue o array e passe como argumento para a função preparada para tal.

Sintaxe:

```
1. function exemplo ([parâmetro1,parâmetro2,parâmetro3]){  
2.   //bloco de código;  
3. }  
4.  
5. const array = [valor1,valor2,valor3];  
6.  
7. exemplo(array);
```

V. Atribuição via destruturação com funções para objetos

Conceito: é possível pegar os atributos de um objeto e atribuí-los aos parâmetros individuais de uma função, basta que eu pegue o objeto e passe como argumento para a função preparada para tal.

Sintaxe:

```
1. function exemplo ({parâmetro1,parâmetro2,parâmetro3}){  
2.   //bloco de código;  
3. }  
4.  
5. const objeto = {  
6.   atributo1:valor1,  
7.   atributo2:valor2,  
8.   atributo3:valor3,  
9. }  
10.  
11. exemplo(objeto);
```

VI. Operador rest com funções.

Conceito: é possível atribuir os primeiros argumento a um certo número de parâmetros individuais e todo o resto a um único parâmetro que irá se comportar como um array.

Sintaxe:

```
1. function exemplo (parâmetro1, parâmetro2, ...parâmetro3){  
2.   bloco de código;  
3. }  
4.  
5. exemplo(valor1, valor2, valor3, valor4, valor5, valorN);
```

3. Retorno de função.

I. Definição

Definição: um retorno normalmente é um valor, podendo ser também uma ação, que uma função entrega como resultado após ser executada.

II. Armazenamento de retorno.

Conceito: ao trabalharmos com funções é normal que o seu retorno seja armazenado dentro de uma variável.

Sintaxe:

```
1. function exemplo(p1,p2,...){  
2.   //bloco de código  
3.   valor de retorno;  
4. }  
5.  
6. const armazena = exemplo(a1,a2,...);
```

III. Função como retorno de função.

Conceito: como função são objetos de primeira classe, elas podem servir de retorno de outras funções. Essas funções retornadas de outras funções acabam herdando os parâmetros da função que está lhe retornando.

Sintaxe:

```
1. function exemplo1(p1,p2,...){
2.   function exemplo2(p1,p2,...){
3.     //bloco de código
4.     valor de retorno;
5.   }
6.   return exemplo2;
7. }
8.
9. const armazenaExemplo2 = exemplo1(a1,a2,...);
10. const armazenaRetornoExemplo2 = armazenaExemplo2(a1,a2,...);
```

4. Escopo léxico

Definição: significa que o escopo de uma entidade (variável, array, funções ou objetos), é definido pelo o local onde ela é declarada no código.

Exemplo 1: Escopo global

```
1. const exemplo1 = "Nome";
2.
3. function teste(){
4.   //bloco de código;
5.   return;
6. }
```

Exemplo 2: Escopo local

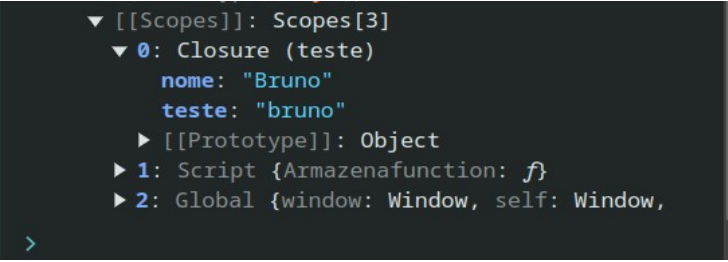
```
1. function teste(){
2.   const exemplo2 = "Nome";
3.   return;
4. }
```

5. Closures.

Definição: closures ocorre quando um função interna tem acesso ao seu próprio escopo, ao escopo de uma função externa e ao escopo global; isso permite que a função interna tenha acesso a três ou mais escopos.

Exemplo: closure

```
1. function externa (teste){
2.   const nome = "Bruno";
3.   return function(){
4.     console.log(nome);
5.     console.log(teste);
6.   }
7. }
8. const saudacao = externa("bruno").
9. saudacao();
```



```
▼ [[Scopes]]: Scopes[3]
  ▼ 0: Closure (teste)
    nome: "Bruno"
    teste: "bruno"
    ► [[Prototype]]: Object
    ► 1: Script {Armazenafunction: f}
    ► 2: Global {window: Window, self: Window,
>
```

7. Funções de callback.

Definição: uma função de callback é uma função passada como argumento para outra função, que será executada depois que essa outra terminar sua tarefa.

Uso: isso é usado principalmente para controlar a ordem da execução de funções, especialmente quando lidamos com tarefas assíncronas, foras de ordem.

Sintaxe:

```
1. function teste(callback){
2.   //bloco de código;
3.   testeCallback();
4. }
5.
6. function testeCallback(){
7.   //bloco de código;
8.   return;
9. }
10.
11. teste(testecallback)
```

8. Funções imediatas(IIFE).

Definição: uma função imediata é uma função anônima que é definida e executada imediatamente, no momento em que o código é interpretado.

Uso: isso é usado principalmente para criar um escopo isolado que evita conflitos de variáveis no escopo global.

Sintaxe:

```
1. (functio(p1,p2,...){  
2.  //bloco de código;  
3. })(a1,a2,...);
```

9. Função fábrica.

I. Definição

Definição: uma função fábrica é uma função que retorna um objeto. Ela “fabrica” um objeto com base na sua estrutura e nos argumentos que recebe.

Sintaxe:

```
1. function fabrica(p1,p2,pn){  
2.   return {  
3.     atributo1: p1,  
4.     atributo2: p2,  
5.     atributon: pn,  
6.  
7.     metodo1(){  
8.       //bloco de código;  
9.       return;  
10.    },  
11.    metodo2(){  
12.      //bloco de código;  
13.      return;  
14.    },  
15.  };  
16. }  
17. const objeto1 = fabrica(a1,ar2,an);  
18. const objeto2 = fabrica(a1,a2,an)
```

II. Método e atributos.

1. Definição

Atributos: são variáveis dentro do objeto que servem para armazenar características deste.

Métodos: são funções dentro do objeto que servem para que o objeto execute alguma ação.

2. Acesso X Execução

Acessando atributos: para acessar um atributo basta dizer o identificador do objeto e o atributo que deseja acessar.

Sintaxe: identificadorObjeto.atributo;

Executando métodos: para executar um método basta dizer o identificador do objeto e o método que deseja executar.

Sintaxe: identificadorObjeto.método();

III. This

Definição: o this é usado especialmente para acessar diretamente os atributos de um objeto de dentro de um método.

Exemplo:

```
1. function fabrica(nome){
2.   return{
3.     nome,
4.     falaNome(){
5.       console.log(`Olá, meu nome é ${this.nome}`);
6.     }
7.   };
8. }
9. const pessoa1 = fabrica("Bruno");
10. pessoa1.falaNome();
```

Explicação: quando pessoa1 chamar o método falaNome, esse irá falar o nome que está armazenado no seu atributo nome, por conta do comando this.nome, é como se desse o comando p1.nome.

IV. Getters e setters.

1. Getters

Definição: no contexto de funções fábrica o comando get permite que métodos de objetos sejam acessados como atributos.

Sintaxe:

```
1. get metodo (){
2.   //bloco de código;
3.   return;
4. }
5. identificadorObjeto.método;
```

2. Setters

Definição: no contexto de funções fábrica o comando set permite modificar, controlar e até validar dados antes de armazená-los.

Sintaxe:

```
1. set metodo(){
2.   //Bloco de código;
3.   return;
4. }
5. identificadorObjeto.metodo = Novo Valor;
```


10. Funções construtoras.

I. Definição.

Definição: funções construtoras são funções utilizadas para criar objetos com atributos e métodos específicos.

Sintaxe:

```
1. function Identificador(p1,p2,n2){  
2.   this.atributo1;  
3.   this.atributo2;  
4.   this.atributon;  
5.  
6.   this.metodo=function(){  
7.     //bloco de código;  
8.     return;  
9.   };  
10. }
```

Observações:

Obs1: identificadores de funções construtoras devem ser declarados com a primeira letra maiúscula.

Obs2: atributos e métodos públicos são antecidos pelo termo “this”.

II. Atributos e métodos privados.

Definição: atributos e métodos privados são elementos da função construtora que não podem e nem devem ser utilizados fora da função.

Sintaxe:

```
1. function Identificador(p1,p2,n2){  
2.   const atributoPrivado; //Atributo privado  
3.   this.atributo1=p1;  
4.   this.atributo2=p2;  
5.   this.atributon=pn;  
6.  
7.   this.metodo=function(){  
8.     //bloco de código;  
9.     return;  
10.  };  
11.  const metodoPrivado = function(){ //Método privado  
12.    //bloco de código;  
13.    return;  
14.  };  
15. }
```

11. Funções recursivas.

Definição: uma função recursiva é uma função que se chama a si mesma. Normalmente ela irá executar algum bloco de código e ao final irá se chamar novamente.

Sintaxe:

```
1. function recursiva(p1,p2,pn){  
2.     bloco de código;  
3.     condição de parada return;  
4.     recursiva();  
5. }
```

Observação: é preciso definir uma condição de parada a função, normalmente é colocado uma estrutura de seleção que, se a condição for verdadeira irá ser executado o return da função fazendo-a parar.

12. Funções geradoras.

I. Definição

Definição: funções geradora são funções especiais que podem pausar sua execução e retomar depois. Elas usam a palavra yield para retornar um objeto e pausar sua execução, após ser chamada novamente continuar sua execução a partir de linha logo abaixo o yield.

Sintaxe:

```
1. function* identificador(p1,p2,pn){  
2.     //Bloco de código;  
3.     yield objeto;  
4.     //Bloco de código  
5.     yield objeto;  
6. }
```

Elementos:

1. **yield:** o yield funciona como o return de uma função comum, porém ele sempre retorna um objeto que possui duas propriedades importante o value e o done.
2. **value:** é o elemento que está sendo retornado.
3. **done:** informa se a execução terminou, com true ou false.
3. **Chamada de uma função geradora:** durante a chamada de uma função geradora é preciso usar o **.next()**; esse comando executa a função de onde ela parou e, se eu quiser ver o valor do objeto que ela está retornando eu uso **od .next().value**.

Sintaxe de chamada: identificador().next().value;

II. Iteração sobre funções geradora com for of.

Conceito: com a estrutura de repetição for of, é possível executar uma função geradora repetidamente e automaticamente,tendo acesso direto aos valores retornado, sem fazer várias declarações no código

Exemplo:

```
1. function* geradora(){
2.   yield "1";
3.   yield "2";
4.   yield "3";
5. }
6.
7. for (let valor of geradora()){
8.   console.log(valor);
9.}
```

III. Loop infinito

Conceito: usando a estrutura de repetição while, é possível criar uma função geradora infinita.

Exemplo:

```
1.function* geradora(){
2.  let i = 0;
3.  while(true){
4.    yield i;
5.    i++;
6.  }
7. }
8.
9. console.log(geradora().next().value);
```

Observação: a cada chamada a função geradora irá iterar no laço pausadamente, sem sair dele.

IV. Função geradora que executa outra função geradora.

Conceito: é possível fazer uma função geradora chamar outra função geradora, assim criando uma ordem na execução de cada uma.

Exemplo:

```
1 .function* geradora1(){
2.   yield "1";
3.   yield "2";
4.   yield "3";
5. }
6.
7. function* geradora2(){
8.   yield* geradora1();
9.   yield "4";
10.  yield "5";
11.  yield "6";
12. }
13. geradora2();
```

V. Função geradora que retorna função.

Conceito: também é possível retornar funções com o yield.

Exemplo:

```
1. function* geradora(){
2.   yield function(){
3.     //bloco de código;
4.     return;
5.   }
6.   yield function(){
7.     //bloco de código;
8.     return;
9.   }
10. }
11. const funcao1 = geradora().next.value;
12. const funcao2 = geradora().next.value;
```

Observação: o yield ainda continua retornando um objeto, só que nesse caso em específico o value é igual a uma função anônima, que está sendo extraída e armazenada nas variáveis.